```
from abc import ABC, abstractmethod
class DoctorNotFoundException(Exception):
    pass
class PatientNotFoundException(Exception):
    pass
class AppointmentAlreadyExistsException(Exception):
    pass
class InvalidAgeException(Exception):
    pass
class InvalidFeeException(Exception):
    pass
class BillingException(Exception):
    pass

class Person(ABC):
    def __init__(self, pid, name, age, phone):
        self.id = pid
        self.name = name

        if age <= 0:
            raise InvalidAgeException("Age must be greater than zero.")

        self.age = age
        self.phone = phone

    @abstractmethod
    def display_details(self):
        pass

class Doctor(Person):
    def __init__(self, doctor_id, name, age,
phone,specialization,consultation_fee):

        super().__init__(doctor_id, name, age, phone)

        if consultation_fee <= 0:
            raise InvalidFeeException("Consultation fee must be greater than
zero.")

        self.doctor_id = doctor_id
        self.specialization = specialization
        self.__consultation_fee = consultation_fee

    def get_consultation_fee(self):
        return self.__consultation_fee

    def set_consultation_fee(self, fee):
        if fee <= 0:
            raise InvalidFeeException("Consultation fee must be greater than
zero")
        self.__consultation_fee = fee

    def display_details(self):
        print("\nDoctor Details")
        print("Doctor ID:", self.doctor_id)
        print("Name:", self.name)
        print("Specialization:", self.specialization)
        print("Consultation Fee:", self.__consultation_fee)

class Patient(Person):
    def __init__(self, patient_id, name, age, phone,
                 disease, admission_status="Admitted"):

        super().__init__(patient_id, name, age, phone)
```

```
        self.patient_id = patient_id
        self.disease = disease
        self.admission_status = admission_status
        self.__medical_history = []

    def get_medical_history(self):
        return self.__medical_history

    def add_medical_history(self, record):
        self.__medical_history.append(record)

    def display_details(self):
        print("\nPatient Details")
        print("Patient ID:", self.patient_id)
        print("Name:", self.name)
        print("Disease:", self.disease)
        print("Status:", self.admission_status)

class HospitalManagement:
    def __init__(self):
        self.doctors = {}
        self.patients = {}
        self.appointments = []
        self.diseases = set()

    def add_doctor(self, doctor):
        if doctor.doctor_id in self.doctors:
            print("Doctor ID already exists.")
            return

        self.doctors[doctor.doctor_id] = doctor
        print("Doctor added successfully.")

    def add_patient(self, patient):
        if patient.patient_id in self.patients:
            print("Patient ID already exists.")
            return

        self.patients[patient.patient_id] = patient

        self.diseases.add(patient.disease)

        print("Patient added successfully.")

    def view_all_doctors(self):

        for doctor in self.doctors.values():
            print(
                f"Doctor ID: {doctor.doctor_id}, "
                f"Name: {doctor.name}, "
                f"Specialization: {doctor.specialization}, "
                f"Consultation Fee: {doctor.get_consultation_fee()}")

    def view_all_patients(self):

        for patient in self.patients.values():
            print(
                f"Patient ID: {patient.patient_id}, "
                f"Name: {patient.name}, "
                f"Disease: {patient.disease}, "
                f"Admission Status: {patient.admission_status}")

    def search_doctor(self, doctor_id):
```

```
        if doctor_id not in self.doctors:
            raise DoctorNotFoundException("Doctor does not exist.")

        self.doctors[doctor_id].display_details()

    def search_patient(self, patient_id):
        if patient_id not in self.patients:
            raise PatientNotFoundException("Patient does not exist.")

        self.patients[patient_id].display_details()

    def book_appointment(self, appointment_id,doctor_id,patient_id,date,time):

        if doctor_id not in self.doctors:
            raise DoctorNotFoundException("Doctor does not exist.")

        if patient_id not in self.patients:
            raise PatientNotFoundException("Patient does not exist.")

        for appointment in self.appointments:
            if (appointment["doctor_id"] == doctor_id and
                    appointment["date"] == date and
                    appointment["time"] == time):
                raise AppointmentAlreadyExistsException("Appointment slot
already booked.")

        appointment = {
            "appointment_id": appointment_id,
            "doctor_id": doctor_id,
            "patient_id": patient_id,
            "date": date,
            "time": time}

        self.appointments.append(appointment)

        print("Appointment booked successfully.")

    def view_appointments(self):
        for appointment in self.appointments:
            doctor_name = self.doctors[appointment["doctor_id"]].name

            patient_name = self.patients[appointment["patient_id"]].name
            print(
                f"Appointment ID: {appointment['appointment_id']}, "
                f"Doctor: {doctor_name}, "
                f"Patient: {patient_name}, "
                f"Date: {appointment['date']}, ")

    def calculate_bill(self, doctor_id):
        try:
            if doctor_id not in self.doctors:
                raise DoctorNotFoundException("Doctor not found.")

            return self.doctors[doctor_id].get_consultation_fee()

        except Exception as e:
            raise BillingException(f"Billing calculation failed: {e}")

hospital = HospitalManagement()

try:

    d1 = Doctor("D403","sahithi",25,"9874563280","Neurology",1500)
    d2 = Doctor("D302","sheela",35,"9576246210","Cardiology",1000)
```

```
    hospital.add_doctor(d1)
    hospital.add_doctor(d2)
    p1 = Patient("P001","rani",24,"9421459780","heart pain")
    p2 = Patient("P002","Rahul",30,"9136568893","fever")
    hospital.add_patient(p1)
    hospital.add_patient(p2)
    p1.add_medical_history("Visited on 11-06-2026")
    p1.add_medical_history("Blood Test Completed")
    hospital.view_all_doctors()
    hospital.view_all_patients()
    hospital.search_doctor("D403")
    hospital.search_patient("P001")
    hospital.book_appointment("A101","D302","P001","13-06-2026","11:00 AM")
    hospital.book_appointment("A102","D403","P002","15-06-2026","09:00 AM")

    hospital.view_appointments()
    print("\nUnique Diseases:", hospital.diseases)
    bill = hospital.calculate_bill("D403")
    print("\nConsultation Bill:", bill)
except (
    DoctorNotFoundException,
    PatientNotFoundException,
    AppointmentAlreadyExistsException,
    InvalidAgeException,
    InvalidFeeException,
    BillingException
) as e:
    print("Error:", e)
```